ASSIGNMENT 2

# Large Language Models, Chatbot Architecture and Vector Databases

---

*A self-assessment and a research study on building LLM-based conversational systems and the retrieval infrastructure behind them.*

Lakshin Khurana

October 19, 2026

# CONTENTS

**1 Question 1: Self-Assessment** **1**
1.1 My Rating . . . . . 1
1.2 Why B and Not A . . . . . 1
1.3 Conceptual Evidence: Relative Self-Attention in My Music Transformer Project . . . . . 2

**2 Question 2: Architecture of an LLM-Based Chatbot** **5**
2.1 Conceptual Understanding . . . . . 5
2.2 Problem Statement . . . . . 5
2.3 Proposed Solution . . . . . 5

**3 Question 3: Vector Databases** **11**
3.1 Conceptual Understanding . . . . . 11
3.2 Problem Statement . . . . . 11
3.3 Proposed Solution . . . . . 13

**4 Conclusion** **17**

**References** **17**

# 1. QUESTION 1: SELF-ASSESSMENT

## *1.1 My Rating*

The scale is: *A* = can code independently, *B* = can code under supervision, *C* = little or no understanding.

TABLE 1. Self-assessment by area.

| *Area* | Rating | Basis for the rating |
|---|---|---|
| *Large Language Models* | B | I understand the transformer, tokenization, prompting, RAG and fine-tuning ideas, and I can build working pipelines with guidance and reference material. |
| *Deep Learning* | B | I understand backpropagation, attention, normalization and training dynamics, and I have implemented sequence models, though not always entirely from a blank file. |
| *Artificial Intelligence* | B | I am comfortable with the broader landscape (search, representation, agents, retrieval) at a conceptual level and apply it in projects. |
| *Machine Learning* | B | I understand the standard supervised workflow, evaluation and overfitting, and can implement solutions with some support. |

## *1.2 Why B and Not A*

I want to be very honest about this part . A lot of my day-to-day workflows depend on me using AI tools to their fullest extent since they are really efficient and allow me to delegate tasks to agents while I do the things they cannot do yet . That makes me productive, but it also means that if I were placed in front of an empty editor with no assistance, I would not reliably produce a correct, optimized implementation of a non-trivial deep learning system on my own .

At the same time , I do not think the rating should be C . If I pay attention to what I am reading , I can catch conceptual mistakes , understand edge cases and metrics to identify what needs to be modified for things to work . This is precisely the profile of a B: I can code under supervision, and my conceptual understanding is stronger than my implementation speed.

> WHAT THIS MEANS IN PRACTICE
>
> I can be given a problem, research it, choose an approach, and deliver a working implementation with access to documentation and tools. I am still building the habit of writing complex model code from scratch without that support, and that is the gap between my current B and an A.

### *1.3 Conceptual Evidence: Relative Self-Attention in My Music Transformer Project*

To show that the B is grounded in real understanding and not just tool usage, I describe one concept I applied in a previous project: a decoder-only Music Transformer, in which I used *relative self-attention*. Below I explain the idea, what I actually implemented, and how my version differs from the original formulation.

**The problem with absolute positions**

Standard self-attention computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{1}$$

which is permutation-invariant. Order is injected by adding *absolute* positional encodings to the input embeddings. For music this is a poor fit. Musical structure is dominated by *relative* relationships: a note is meaningful because of how far it is, in time, from a previous beat, a repeated motif, or the start of a bar. A motif played at position 40 and again at position 400 is the same motif, but absolute encodings present it to the model in two unrelated ways.

**Relative position information in the literature**

Shaw et al. [2] made the attention score between positions $i$ and $j$ depend on the offset $j - i$. The Music Transformer of Huang et al. [3] adapts this to music with

$$e_{ij} = \frac{q_i k_j^\top + q_i r_{j-i}^\top}{\sqrt{D_h}}, \qquad S^{\text{rel}} = \text{skew}(QR^\top), \tag{2}$$

where $R \in \mathbb{R}^{L \times D_h}$ stores one learned embedding per relative distance. A naive implementation materializes a tensor of shape $L \times L \times D_h$, which costs $O(L^2 D)$ memory. The "skewing" step avoids this: $Q$ is multiplied directly with $R^\top$, giving a matrix indexed by relative distance, which is then converted to absolute indexing by padding a dummy column, reshaping, and slicing. This brings the intermediate memory down to $O(LD)$. A simpler alternative, used for example in T5 [**?**], replaces the embedding vectors with a single learned scalar bias per offset.

**What I implemented**

My implementation follows the simpler scalar-bias route. Each attention head owns a learnable vector $b \in \mathbb{R}^{2L-1}$, holding one scalar for every offset in $\{-(L-1), \ldots, L-1\}$, where $L$ is the context length. For a sequence of length $T \leq L$, the logit between positions $i$ and $j$ is

$$e_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}} + b_{j-i} \quad (j \leq i), \qquad e_{ij} = -\infty \quad (j > i), \tag{3}$$

followed by a softmax over $j$. In code, this is done in five steps:

(1) Build a $T \times T$ matrix of offsets $j - i$ from two index ranges.

(2) Clamp the offsets to $[-(L-1),\ L-1]$ and shift them by $L-1$ so that they are valid non-negative indices.
(3) Look up $b$ with these indices to obtain a $T \times T$ bias matrix.
(4) Add this matrix to the scaled dot-product scores, broadcasting over the batch dimension.
(5) Apply the causal mask and the softmax.

**How my version differs from the Music Transformer**

TABLE 2. The original relative attention formulation compared with my simplified version.

| | Music Transformer (Huang et al.) | My implementation |
|---|---|---|
| Relative term | $q_i r_{j-i}^{\top}$, a learned vector per offset | $b_{j-i}$, a learned scalar per offset |
| Depends on query content | Yes | No |
| Parameters per head | $L \times D_h$ | $2L-1$ |
| Extra memory | $O(LD)$ with skewing | One $T \times T$ index lookup |
| Skewing required | Yes | No |

Because the bias is a scalar, no $L \times L \times D_h$ tensor is ever created, so there is nothing for the skewing step to fix. The only extra cost is one $T \times T$ index lookup, which is of the same order as the attention matrix itself. The price is expressiveness: the bias does not depend on the query, so the model cannot decide that a note four steps back matters for one kind of query and not for another. Each head can still learn its own offset preference, which is why I used one bias vector per head.

The reason I chose this approach rather than the standard approach given in the original Music Transformer was due to resource constraints on cloud computing services such as Google Colab but this architecture still allows me to build a relatively naive version of the original research .

**What I took from the project**

- Replacing absolute positional encodings with relative ones is not a cosmetic change; it changes what the model can generalize over, because rhythm and repetition are translation-invariant.
- Efficiency is part of the architecture. The skewing trick in the original paper exists because vector-valued relative embeddings are expensive, and my scalar bias is a simplification that avoids that cost.
- Under causal masking, only the non-positive offsets ever receive gradient, so roughly half of the $2L-1$ bias entries are unused. A vector of length $L$ would be enough.
- I understood this mostly by reading the papers and working through the shapes with tool assistance, which is exactly why I place myself at B: strong on the *why*,

still developing on writing every piece unaided.

# 2. QUESTION 2: ARCHITECTURE OF AN LLM-BASED CHATBOT

## *2.1 Conceptual Understanding*

**What the LLM can and cannot do alone**

A large language model is a transformer-based next-token predictor trained on a very large corpus. On its own it has three structural limitations that shape every chatbot design:

(1) *Frozen knowledge.* It knows only what was in its training data, up to a cutoff date, and knows nothing about private or recent information.
(2) *Hallucination.* It produces fluent text whether or not it is correct, and has no built-in notion of "I do not have a source for this".
(3) *Finite context and statelessness.* Each call sees only the tokens in its context window; any conversational memory must be supplied explicitly.

A production chatbot is therefore not "an LLM with a text box". It is a system built *around* the model to supply knowledge, memory, safety and measurement.

**Three ways to give an LLM knowledge**

In my understanding, RAG is the default choice for most knowledge-grounded chatbots, and fine-tuning is a complement for behaviour rather than a replacement for retrieval.

## *2.2 Problem Statement*

> *Hypothetical problem.* A B2B software company has roughly 2,000 pages of product documentation, API references and policy articles, which change weekly. It wants a customer-support chatbot that (i) answers questions *grounded in these documents*, (ii) cites the source of each answer, (iii) refuses or escalates to a human when the documents do not support an answer, (iv) handles multi-turn conversations, and (v) keeps one customer's private data invisible to another.

## *2.3 Proposed Solution*

**High-level architecture**

The design has an *offline* ingestion path that prepares knowledge, and an *online* serving path that answers each message. Figure 1 summarizes the online path and its supporting components.

**Component by component**

**Walkthrough of one request**

(1) The user sends a message. Input guardrails strip or flag personal data and detect attempts to override the system instructions.
(2) The query is rewritten into a standalone form using the stored conversation memory.

TABLE 3. Approaches for supplying knowledge to an LLM.

| *Approach* | Strength | Weakness | Best used when |
|---|---|---|---|
| *Prompting with long context* | Simplest, no infrastructure | Costly per call, degrades on very long inputs, no scaling to large corpora | Small, static knowledge |
| *Fine-tuning* | Changes style, format and behaviour | Poor at injecting facts, expensive to update, no citations | Tone, domain language, structured output |
| *Retrieval-augmented generation (RAG) [4]* | Fresh, citable, updatable without retraining | Quality bounded by retrieval; added latency and moving parts | Large or changing knowledge, need for citations |

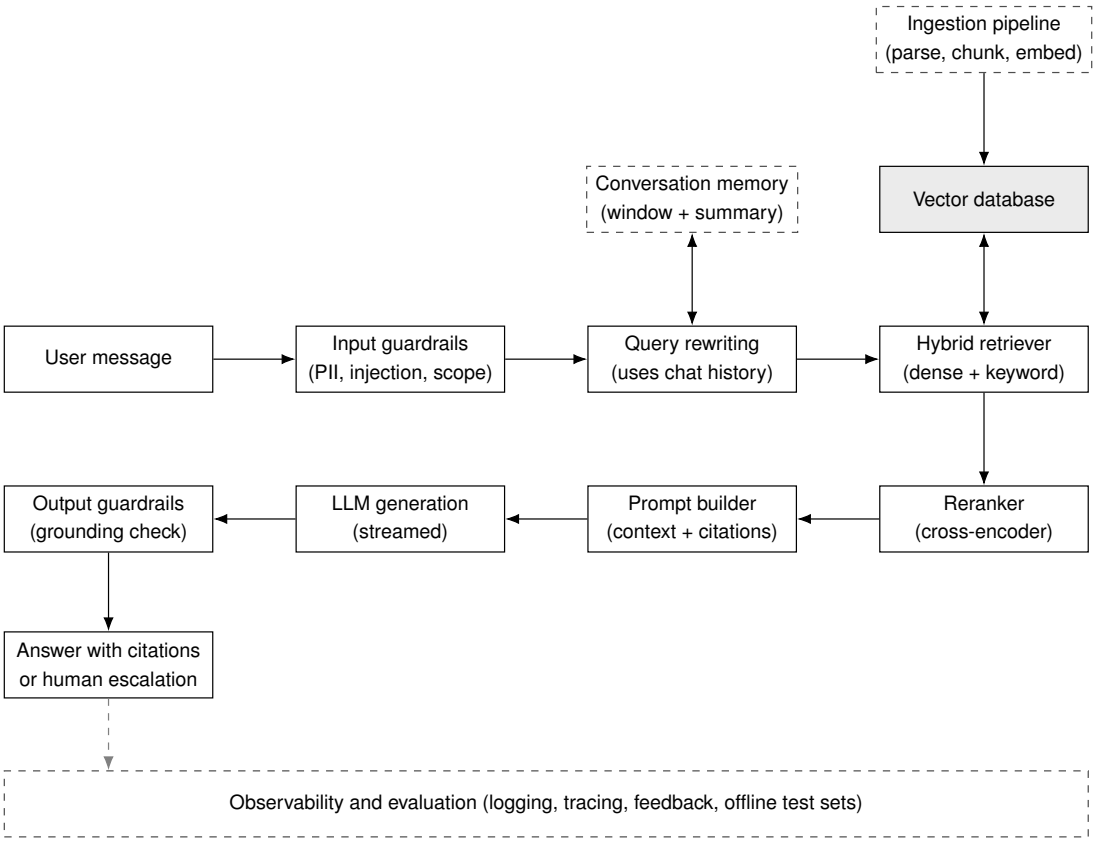



FIGURE 1. Online serving path of the proposed RAG chatbot, with its supporting stores.

TABLE 4. Key architectural components, their responsibilities and failure modes.

| *Component* | Responsibility | Main failure mode to watch |
|---|---|---|
| *Ingestion pipeline* | Parse documents, clean them, split into chunks, attach metadata (tenant, version, access role), embed and upsert. | Bad parsing of tables or code; stale chunks after a document is updated. |
| *Chunking* | Split by document structure (headings, paragraphs) into pieces of a few hundred tokens with modest overlap. | Chunks too small lose context; too large dilute the embedding. |
| *Embedding model* | Map chunks and queries into the same vector space. | Domain mismatch; changing the model forces re-embedding everything. |
| *Vector database* | Store vectors and metadata; serve filtered approximate nearest-neighbour search. | Recall loss from aggressive approximation; missing filters leak data. |
| *Query rewriting* | Turn a follow-up such as "and what about the limit?" into a standalone query using the history. | Over-rewriting that changes the user's intent. |
| *Hybrid retriever* | Combine dense similarity with keyword (BM25-style) matching, merged by rank fusion. | Dense-only retrieval misses exact identifiers, error codes and names. |
| *Reranker* | Re-score the top candidates with a more accurate cross-encoder and keep the best few. | Added latency; must be limited to a small candidate set. |
| *Prompt builder* | Assemble system instructions, retrieved passages with source ids, history and the question. | Prompt injection through retrieved text; overflowing the context window. |
| *LLM* | Generate an answer using *only* the supplied context, in a defined format with citations. | Ignoring context and answering from parametric memory. |
| *Guardrails* | Screen inputs for PII and injection attempts; check that the output is supported by the context. | False refusals; checks that are too weak to catch unsupported claims. |
| *Memory* | Keep recent turns verbatim and summarize older ones. | Summaries that silently drop important constraints. |
| *Observability and evaluation* | Trace every request, collect feedback, and run regression tests. | Optimizing for demos instead of measured quality. |

(3) The hybrid retriever fetches, for example, the top 30 candidate chunks, filtered by the user's tenant and access role, from the vector database.
(4) The reranker re-scores these candidates and keeps the best five to eight.
(5) The prompt builder assembles the instructions, the selected passages (each tagged with a source id), the recent history and the question.
(6) The LLM generates the answer, instructed to cite passage ids and to say that it does not know when the context is insufficient. The response is streamed to reduce perceived latency.
(7) Output guardrails verify that cited claims are supported by the passages. If not, the system falls back to a safe refusal or hands the conversation to a human agent.

**Evaluation**

A chatbot cannot be improved reliably without measurement. I would evaluate the retrieval and generation stages separately, because a wrong answer can originate in either.

TABLE 5. Evaluation dimensions for a RAG chatbot.

| *Stage* | Metrics | Question answered |
|---|---|---|
| *Retrieval* | Recall@k, MRR, context precision | Did the right passage reach the model? |
| *Generation* | Faithfulness (groundedness), answer relevance | Is the answer supported by the passages and on topic? |
| *System* | Latency (p50 and p95), cost per conversation, escalation rate, user feedback | Is it fast, affordable and actually useful? |

A fixed set of representative questions with reference answers is run on every change to chunking, embedding model, prompt or LLM, so that regressions are visible before release.

**Design decisions and trade-offs**

– *RAG over fine-tuning* because the documents change weekly, answers must be citable, and updating an index is far cheaper than retraining.
– *Hybrid retrieval* because support questions often contain exact tokens (error codes, endpoint names) that pure semantic search handles poorly.
– *A reranking stage* because first-stage vector search optimizes speed, and a cross-encoder over a short list recovers precision at modest cost.
– *Refusal as a feature.* For a support product, a confident wrong answer is worse than escalation, so the system is designed to abstain when evidence is weak.
– *Cost and latency controls*: response streaming, caching of frequent queries, and a smaller model for query rewriting with a stronger one reserved for final genera-

tion.

# 3. QUESTION 3: VECTOR DATABASES

## *3.1 Conceptual Understanding*

**Embeddings and similarity**

An embedding model maps a piece of data (text, an image, audio) to a point in a high-dimensional vector space so that semantically similar items lie close together. A vector database is a system purpose-built to store such vectors together with their metadata and to answer the question: *given a query vector, which stored vectors are nearest to it?* Closeness is measured with a metric:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}, \qquad d_2(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2, \tag{4}$$

and for normalized vectors, cosine similarity and the dot product give identical rankings.

**Why not just use a normal database?**

An exact search compares the query against every stored vector, costing $O(Nd)$ per query. With ten million vectors of dimension 1024, that is on the order of $10^{10}$ multiplications for a single query, which is too slow for interactive use. Vector databases therefore rely on *approximate nearest neighbour* (ANN) indexes, which trade a small, tunable loss of recall for orders of magnitude lower latency.

**Core indexing ideas**

**What distinguishes a database from a library**

A library such as FAISS [7] provides the index algorithms. A *database* adds what production requires: persistence, create/update/delete operations, *metadata filtering*, replication and sharding, access control, backups and an API. In RAG systems the filtering capability is often the deciding factor, because queries are almost never "nearest vectors overall" but rather "nearest vectors *belonging to this tenant, in the current document version, that this user is allowed to see*".

## *3.2 Problem Statement*

> *Hypothetical problem.* The support chatbot of Question 2 has grown into a multi-tenant product serving about 200 business customers.
>
> – *Scale:* about 10 million text chunks, embedded at 1024 dimensions.
> – *Filtering:* every query must be restricted by tenant, document version and user access role.
> – *Search quality:* hybrid retrieval, combining dense vectors with keyword matching.
> – *Performance:* about 50 queries per second at peak, with retrieval p95 latency under 100 ms.
> – *Team and budget:* two engineers, a limited infrastructure budget, and a requirement that customer data stays inside the company's own cloud account.

TABLE 6. Common ANN and compression techniques.

| *Technique* | Idea | Strength | Cost |
|---|---|---|---|
| *HNSW [5]* | A layered proximity graph; search descends from a sparse top layer to a dense bottom layer. | Excellent recall and speed. | High memory; slower to build. |
| *IVF* | Cluster vectors; search only the nearest few clusters. | Lower memory, fast build. | Recall depends on number of probed clusters. |
| *Product quantization [6]* | Compress vectors into short codes of sub-vector centroids. | Large memory savings. | Lossy; reduces accuracy. |
| *Scalar / binary quantization* | Store each dimension in 8 bits, or in 1 bit. | Large memory savings with small recall loss when combined with rescoring. | Needs a rescoring step for best quality. |

**Sizing the problem**

Raw storage for 10 million float32 vectors of dimension 1024 is $10^7 \times 1024 \times 4$ bytes $\approx$ 41 GB, before index overhead. Figure 2 shows how compression changes this requirement, which matters because fast search wants the working set in RAM.

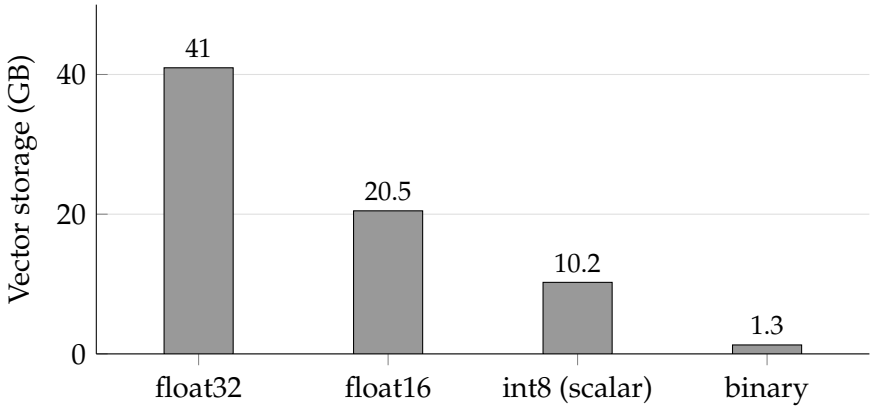



FIGURE 2. Approximate raw storage for $10^7$ vectors at 1024 dimensions under different precisions (excluding index overhead).

## *3.3 Proposed Solution*

**Candidates considered**

**Decision and justification**

> SELECTED: QDRANT
> I would choose *Qdrant*, self-hosted in the company's own cloud account.

The reasons follow directly from the requirements:

(1) *Filtered search is central.* Tenant isolation and access control are not optional here. Qdrant evaluates payload filters during graph traversal, using indexed payload fields, rather than filtering a fixed list of results afterwards, which would return too few results when a filter is selective.

(2) *Memory fits the budget.* Scalar quantization can reduce 41 GB of vectors to roughly 10 GB, with the original vectors kept on disk for rescoring the top candidates. This allows a modest machine to serve the workload at low latency.

(3) *Hybrid retrieval.* Support for sparse vectors alongside dense vectors allows keyword-style matching and semantic matching to be fused in one system.

(4) *Data stays in-house.* It can be self-hosted, satisfying the privacy requirement, and its operational footprint is small enough for two engineers.

(5) *Horizontal growth path.* Collections can be sharded and replicated if the data or traffic outgrows a single node.

TABLE 7. Comparison of candidate vector databases against the stated problem.

| *Option* | Strengths | Limitations for this problem | Verdict |
|---|---|---|---|
| *FAISS* | Very fast, flexible index library. | Not a database: no native filtering, persistence or CRUD. | Rejected |
| *Chroma* | Very easy for prototypes. | Less suited to heavy multi-tenant production load at this scale. | Prototype only |
| *pgvector* | Lives inside PostgreSQL; transactional and familiar. | Filtered ANN search at 10M vectors and 50 QPS needs careful tuning. | Strong runner-up |
| *Pinecone* | Fully managed, minimal operations. | Data leaves our cloud account; recurring cost grows with scale. | Rejected on privacy |
| *Weaviate* | Built-in hybrid search and modules. | Heavier to operate and tune. | Viable |
| *Milvus* | Very high scalability, rich index choices. | Complex distributed deployment for a two-person team. | Overbuilt |
| *Qdrant* | Payload-indexed filtering inside the HNSW search, hybrid support, quantization, simple deployment. | Smaller ecosystem than the largest players. | *Selected* |

TABLE 8. Proposed Qdrant configuration for the multi-tenant knowledge assistant.

| *Aspect* | Choice | Rationale |
|---|---|---|
| *Distance metric* | Cosine | Matches how the embedding model is trained to be compared. |
| *Index* | HNSW with moderate connectivity and a high build-time search depth | Strong recall; tuned against a labelled query set. |
| *Compression* | Scalar (int8) quantization kept in RAM, originals on disk, with rescoring of the top candidates | Cuts memory roughly fourfold with minimal recall loss. |
| *Payload indexes* | Tenant id, document version, access role | Makes filtered search fast and enforces isolation on every query. |
| *Tenancy model* | One shared collection partitioned by a tenant payload field | Avoids hundreds of small collections and their overhead. |
| *Hybrid search* | Dense vector plus sparse keyword vector, fused by rank fusion | Handles both paraphrases and exact identifiers. |

**Proposed configuration**

**Validation plan**

The choice should be verified empirically rather than assumed. I would build a labelled set of questions with known relevant chunks and measure recall@10 against exact search as ground truth, then tune the index search depth until recall is acceptable while keeping p95 latency under 100 ms, under a realistic load test that includes the tenant filters. If quantization causes an unacceptable drop in recall, I would raise the number of rescored candidates before abandoning compression.

**When I would choose differently**

- *Under about one million vectors and an existing PostgreSQL stack:* `pgvector`, to avoid running another system.
- *No appetite for operations and no data residency constraint:* a managed service such as Pinecone.
- *Billions of vectors and a dedicated platform team:* Milvus, for its distributed scalability.

## 4. CONCLUSION

Across the three questions, one theme recurs: models are only one part of a working AI system. Question 1 reflects where I honestly stand, which is a B with solid conceptual grounding that I demonstrated through relative self-attention in my Music Transformer project. Question 2 shows that a reliable chatbot depends on retrieval, grounding, guardrails and evaluation as much as on the language model itself. Question 3 shows that the retrieval layer is an engineering decision shaped by scale, filtering, privacy and team capacity, and that the right vector database is the one that fits the problem rather than the one that is most popular.

## REFERENCES

[1] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] P. Shaw, J. Uszkoreit and A. Vaswani, "Self-Attention with Relative Position Representations," *NAACL-HLT*, 2018.

[3] C.-Z. A. Huang et al., "Music Transformer: Generating Music with Long-Term Structure," *International Conference on Learning Representations (ICLR)*, 2019.

[4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.

[5] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[6] H. Jégou, M. Douze and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011.

[7] J. Johnson, M. Douze and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, 2019.